

Modularización

Tema 3

ÍNDICE

01 Programación modular:funciones

02 Funciones void

03 Diseño de funciones

04 Funciones recursivas

MOTIVACIÓN

- Nuestros programas son pequeños: nos basta el main()
- En la vida real, los programas constan de miles de líneas de código: los dividimos en **módulos**
- Cada **módulo** resuelve una parte del problema
- La unión de los módulos (subproblemas) dan solución al problema completo





“Los problemas complejos se realizan subdividiéndolos en subproblemas más sencillos. La solución cada subproblema es un *módulo* y la unión de los *módulos* dará la solución buscada.”

—A. Garrido

01

FUNCIONES

Funciones en C

- Las funciones de C son la herramienta más básica para crear módulos

- Los módulos son una herramienta fundamental en programación → atacar problemas complejos dividiéndolos en subtareas

- Hemos usado funciones predefinidas (ya creadas): **sqrt(), abs(), round(), etc.**

Ventajas de las funciones

Funciones

Realizar la raíz cuadrada en una función **sqrt()** tiene ventajas



Desarrollo

los subproblemas que necesitan una raíz cuadrada son más sencillos de resolver.

Reusabilidad

una vez resuelta la raíz cuadrada, la podemos usar muchas veces

Mantenimiento

podemos comprobar sqrt() aisladamente. Si no funciona correctamente, sólo hay que “arreglar” el programa en un punto (la función sqrt()).

Funciones vs. No funciones

Sin funciones

```
int numero,i;
int raiz = sqrt(numero);
bool esprimo = true;
for(numero = 2; numero <= n; numero++){
    esprimo = true;
    for(i = 2; i <= raiz && esprimo; i++){
        if(numero % i == 0){
            esprimo = false;
        }
    }
    if(esprimo){
        printf("El numero %d es primo\n",numero);
    }
}
```

Con funciones

```
for(numero = 2; numero <= n; numero++){
    if(EsPrimo(numero)){
        printf("El numero %d es primo\n",numero);
    }
}
```

Definición de funciones C

```
tipo_funcion nombre_funcion(parametros formales){
    ...código de la funcion (cuerpo) ...
}
```

- Creamos una función llamada `nombre_función`
- Encapsula el código en el *cuerpo* de la función

Ejemplo: la función *EsPrimo*

```
tipo_funcion nombre_funcion(parametros formales){
    ...código de la funcion (cuerpo) ...
}
```

```
bool EsPrimo(int n){
    int i,raiz = sqrt(n);
    bool esprimo = true;
    for(i = 2; i <= raiz && esprimo; i++){
        if(n % i == 0){
            esprimo = false;
        }
    }
    return esprimo;
}
```

- La **cabecera** especifica cómo utilizar la función:
- El tipo devuelto al acabar la función `tipo_funcion`

Ejemplo: la función *EsPrimo*

```
tipo_funcion nombre_funcion(parametros formales){
    ...código de la funcion (cuerpo) ...
}
```

```
bool EsPrimo(int n){
    int i,raiz = sqrt(n);
    bool esprimo = true;
    for(i = 2; i <= raiz && esprimo; i++){
        if(n % i == 0){
            esprimo = false;
        }
    }
    return esprimo;
}
```

- La **cabecera** especifica cómo utilizar la función:
- `nombre_funcion` identifica a la función → coincide con la llamada *EsPrimo(numero)* del ejemplo
- Debe seguir las reglas de nombrado de las variables

Ejemplo: la función *EsPrimo*

```
tipo_funcion nombre_funcion(parametros formales){
    ...código de la funcion (cuerpo) ...
}
```

```
bool EsPrimo(int n){
    int i,raiz = sqrt(n);
    bool esprimo = true;
    for(i = 2; i <= raiz && esprimo; i++){
        if(n % i == 0){
            esprimo = false;
        }
    }
    return esprimo;
}
```

- La **cabecera** especifica cómo utilizar la función:
- `parametros` *formales* es una serie de declaración de variables *separadas por comas*
- Identifica los datos que la función necesita para hacer su cometido

Ejemplo: la función *EsPrimo*

```
tipo_funcion nombre_funcion(parametros formales){
    ...código de la funcion (cuerpo) ...
}
```

```
bool EsPrimo(int n){
    int i,raiz = sqrt(n);
    bool esprimo = true;
    for(i = 2; i <= raiz && esprimo; i++){
        if(n % i == 0){
            esprimo = false;
        }
    }
    return esprimo;
}
```

- El código de la función es el conjunto de sentencias que implementan el módulo
- Está delimitado por un par de llaves { y }
- Es un *subprograma* que sólo conoce las variables declaradas dentro (*int* raiz, *bool* esprimo) y los parámetros formales (*int n*)
- Termina con la sentencia return que indica el valor a devolver

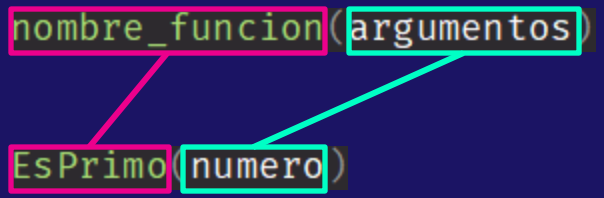
Ejemplo: la función *EsPrimo*

```
tipo_funcion nombre_funcion(parametros formales){
    ...código de la funcion (cuerpo) ...
}
```

```
bool EsPrimo(int n){
    int i,raiz = sqrt(n);
    bool esprimo = true;
    for(i = 2; i <= raiz && esprimo; i++){
        if(n % i == 0){
            esprimo = false;
        }
    }
    return esprimo
}
```

- El tipo de la variable usada en la sentencia `return` debe **coincidir** con el `tipo_funcion` devuelto

Llamando a una función en C



- Utilizamos el `nombre_función` en nuestro programa
- Los `argumentos` son una lista de parámetros, con correspondencia 1 a 1 con el orden establecido en la definición de la función

Primer programa con funciones

- Para poder utilizar una función, C debe “conocerla” antes
- En nuestro ejemplo, si utilizamos la función **EsPrimo** en el main, la función EsPrimo debe incluirse **ANTES** del main
- ¿Cómo podemos utilizar una función sin conocer los detalles de su implementación (cuerpo)?

```c
#include <stdio.h>
#include <math.h>
//Añadir aquí el #define para bool
bool EsPrimo(int n) {
    int i,raiz = sqrt(n);
    bool esprimo = true;

    for (i = 2; i <= raiz && esprimo; ++i) {
        if (n % i == 0)
            esprimo = false;
    }
    return esprimo;
}
int main() {
    int n, numero;

    printf("Introduce un número: ");
    scanf("%d", &n);
    for (numero = 2; numero <= n; numero++) {
        if (EsPrimo(numero))
            printf("%d\n", numero);
    }
    return 0;
}
```

El prototipo de una función

- El prototipo de la función es su *definición* o *declaración*
- Con el **prototipo**, el compilador conoce lo necesario para poder utilizar la función, aunque desconozca su **cuerpo** (implementación).
- El prototipo es la cabecera utilizada en la declaración de la función.
- Prototipo de la función EsPrimo:

```
bool EsPrimo(int n);
```

- Nótese que el prototipo termina en punto y coma

El prototipo de una función

- Con el **prototipo**, el compilador conoce lo necesario para que podamos utilizar la función, aunque desconozca su **cuerpo** (implementación).
- **Ejemplos de prototipos para funciones sencillas:**

```
double Promedio(double v1, double v2);
bool EsNroPar(int nro);
double sqrt(double x);
char toupper(char c);
bool EsMayor(int a, int b);
bool Aprobado(int notaPromedio);
```

- **¿Cómo utilizaríamos estas funciones en nuestro main() ? ¿Cómo implementaríamos el cuerpo de estas funciones?**

Implementación usando el prototipo

```
double Promedio(double v1, double v2){
    return((v1+v2)/2.0);
}

bool EsNroPar(int nro){
    if(nro%2 == 0)
        return(true);
     else
        return(false);
    }
}

bool EsMayor(int a, int b){
    return(a > b);
}
```



```
bool EsNroPar(int nro){
    return(nro%2 == 0);
}
```

¿Cuál es el prototipo?

```
int Sumatoria(int inicio, int fin){
    int suma,i;
    suma = 0;
    for(i=inicio;i <= fin; i=i+1)
        suma = suma + i;
    return(suma);
}

int Potencia(int n, int k){
    int acum,i;
    acum = 1;
    for(i=1;i <= k; i=i+1)
        acum = acum * n;
    return(acum);
}
```

```c
#include <stdio.h>


double PrecioFinal(double costoUnitario, int cantItems);

int main() {
    double costo, precio;
    int cantidad;

    printf("Ingrese la cantidad de items a comprar: ");
    scanf("%d", &cantidad);
    printf("Ingrese el costo por unidad: ");
    scanf("%lf", &costo);

    precio = PrecioFinal(costo, cantidad);

    printf("Total a Pagar (iva incluido): %.2lf\n", precio);
    return 0;
}
```

```c
double PrecioFinal(double costoUnitario, int cantItems){
 const double IVA = 0.21;
 double subTotal;

 subTotal = costoUnitario * cantItems;

 return (subTotal + subTotal*IVA);
}
```

Parámetros Formales

Declaración de la función o prototipo

Llamada a la función

Cabecera de la función

Cuerpo de la función

Traza del programa

```c
int main() {
    double costo, precio;
    int cantidad;

    printf("Ingrese la cantidad de ítems a comprar: ");
    scanf("%d", &cantidad);
    printf("Ingrese el costo por unidad: ");
    scanf("%lf", &costo);

    precio = PrecioFinal(costo, cantidad);

    printf("Total a Pagar (iva incluido): %.2lf\n", precio);
    return 0;
}
```

"Aparcado"

Al llegar a la *llamada*, se COPIAN los valores de *costo* y *cantidad*

El valor devuelto por el *return* se asigna a *precio*

```c
double PrecioFinal(double costoUnitario, int cantItems){
 const double IVA = 0.21;
 double subTotal;

 subTotal = costoUnitario * cantItems;

 return (subTotal + subTotal*IVA);
}
```

Los valores copiados se "pegan" en *costoUnitario* y *cantItems*

Se ejecuta el contenido de la función, utilizando las variables declaradas en ella y los argumentos formales

Parámetros actuales → formales

```
precio = PrecioFinal(5.0, 8 );
```

```
double PrecioFinal(double costoUnitario, int cantItems){
 const double IVA = 0.21;
 double subTotal;

 subTotal = costoUnitario * cantItems;

 return (subTotal + subTotal*IVA);
}
```

Los parámetros actuales se **COPIAN** en los parámetros formales

Se copian en el mismo orden que aparecen en la llamada

Ámbito de las variables (1)

```c
int main() {
    double costo, precio;
    int cantidad;

    printf("Ingrese la cantidad de items a comprar: ");
    scanf("%d", &cantidad);
    printf("Ingrese el costo por unidad: ");
    scanf("%lf", &costo);

    precio = PrecioFinal(costo, cantidad);

    printf("Total a Pagar (iva incluido): %.2lf\n", precio);
    return 0;
}
```

- Las variables *costoUnitario* y *cantItems* no pueden usarse en el main

```c
double PrecioFinal(double costoUnitario, int cantItems){
 const double IVA = 0.21;
 double subTotal;

 subTotal = costoUnitario * cantItems;

 return (subTotal + subTotal*IVA);
}
```

- Las variables *costo* y *cantidad* no pueden usarse en la función `PrecioFinal`
- Sólo los parámetros formales y variables declaradas en la función están en su ámbito

Ámbito de las variables (2)

- Dos funciones diferentes, pueden utilizar los **mismos nombres de variables** sin "interferencias" ya que se refieren a variables **diferentes**

```c
int main (){
    int k = 3;
    printf("%d\n", Cuadrado(k));
    printf("%d\n", k);
    return(0);
}
```

```c
int Cuadrado(int n){
    int k;
    k = n*n;
    return(k);
}
```

La variable *k* sigue valiendo 3 tras la llamada a `Cuadrado(k)`

Ámbito de las variables (3)

- El ámbito está definido por el bloque de código.
- Un bloque de código está delimitado por las llaves '{' y '}'
- El ámbito de la variable está delimitado por el bloque al que pertenece

En un bucle controlado por condición, la variable pierde el valor antiguo en cada iteración:

```
while (condicion){
 int i;
 i = 0;
 ............
}
```

En un bucle for, la variable conserva el valor de la última iteración:

```
for (i=0; i<10; i++){
   ............
}
```

Sólo se asigna i = 0 en la primera iteración.

Funciones que usan funciones

```c
int Cuadrado(int n);
int Cubo(int n);

int main(){
    int k = 3;

    printf("%d\n", Cuadrado(k));
    printf("%d\n", Cubo(k));

    return 0;
}
```

```c
int Cuadrado(int n){
    return(n*n);
}
```

```c
int Cubo(int n){
    return(n*Cuadrado(n));
}
```

La pila

- La llamada a una función necesita reservar espacio en la memoria (en el *stack*)
- Cada llamada genera un bloque de trabajo asociado a ella, (el marco de pila o *stack frame*)
- En el marco asociado a cada función se almacenan, entre otras cosas:
  - Los parámetros formales.
  - Los datos locales (constantes y variables).
  - La dirección de retorno de la función.

La pila

- Cuando una función llama a otra, el bloque o marco de la función llamada se apila sobre el marco de la función desde donde se hace la llamada
- Hasta que no termine de ejecutarse la última función “apilada”, el control no volverá a la anterior.

La pila

- La pila organiza los *stack frames* de una forma muy concreta
- Sólo acepta 2 operaciones para almacenar u obtener *stack frames*
    1. Añadir un stack frame → se almacena "encima" del último dato disponible
    2. Obtener un stack frame → se obtiene el último elemento almacenado y se retira de la pila
- No es posible acceder a elementos que no sean el último apilado
- La posición donde está el elemento accesible es el tope de pila

La pila del sistema

- La “base” de la pila comienza en una posición de la memoria asignada al programa
- Facilita la gestión de las llamadas a las funciones
- Consideremos el siguiente código →

```c
#include <stdio.h>

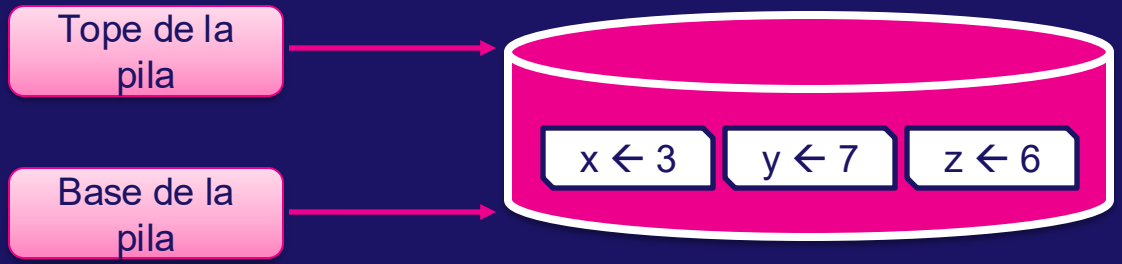
int Maximo(int a, int b){
    if(a > b) return a;
    else return b;
}

int Maximo3(int a, int b, int c){
    int m1 = Maximo(a,b);
    int m2 = Maximo(m1,c);
    return m2;
}

int main(){
    int x, y, z;
    scanf("%d %d %d", &x, &y, &z);
    printf("%d\n", Maximo3(x, y, z));
}
```

Estado de la pila inicial

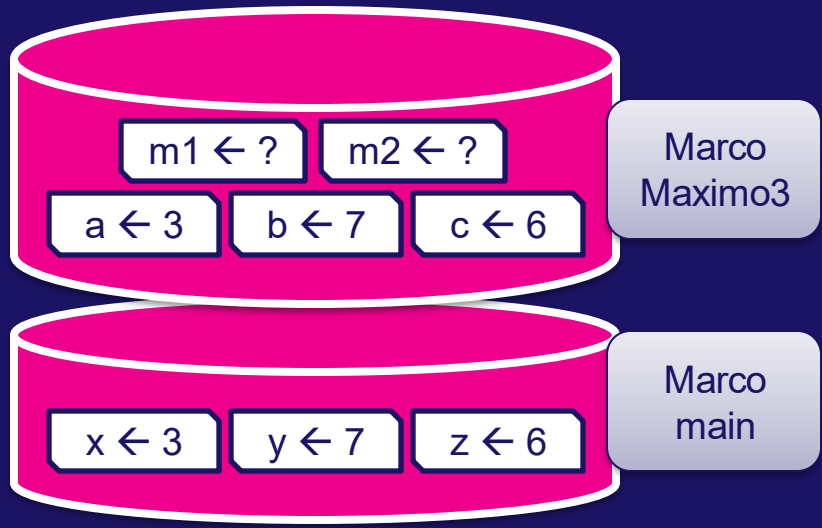
- Supongamos que introducimos 3, 7 y 6 en las variables `x, y, z`



- Antes de la llamada, el sistema reserva espacio para los 3 enteros (x,y,z) y coloca en la pila el segmento del main

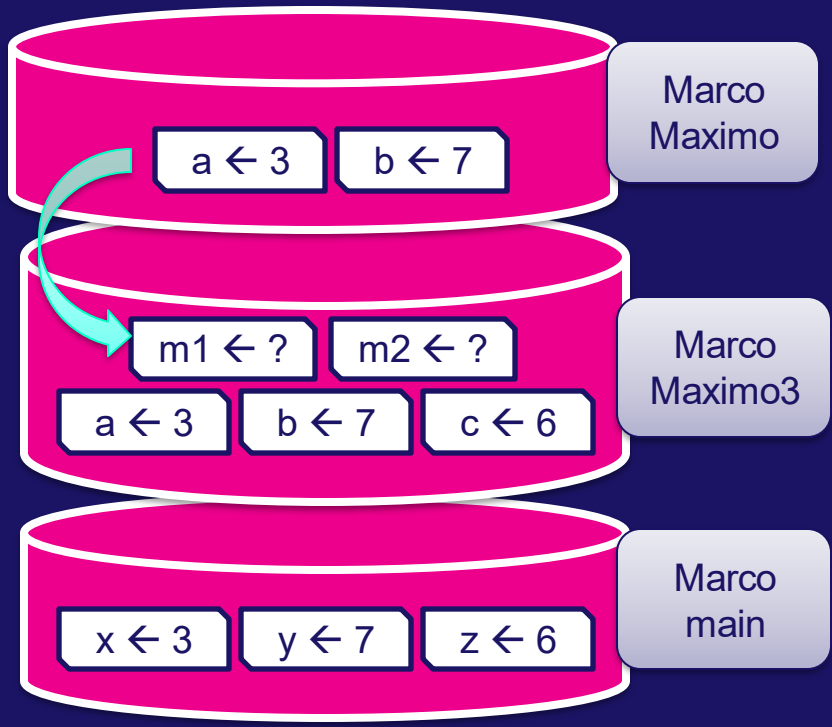
Primera llamada a Maximo3

- En el momento de la llamada a `Maximo3`, el sistema crea un nuevo marco en la pila para alojar el segmento de `Maximo3`
- Si nos damos cuenta, ahora el tope de pila es el segmento de `Maximo3` y las variables x,y,z ya no están en el segmento (o contexto) activo
  - ¡Es por esta razón que no podemos utilizar las variables de otras funciones!
  - Los valores de x,y,z se han copiado en a,b,c en el nuevo contexto

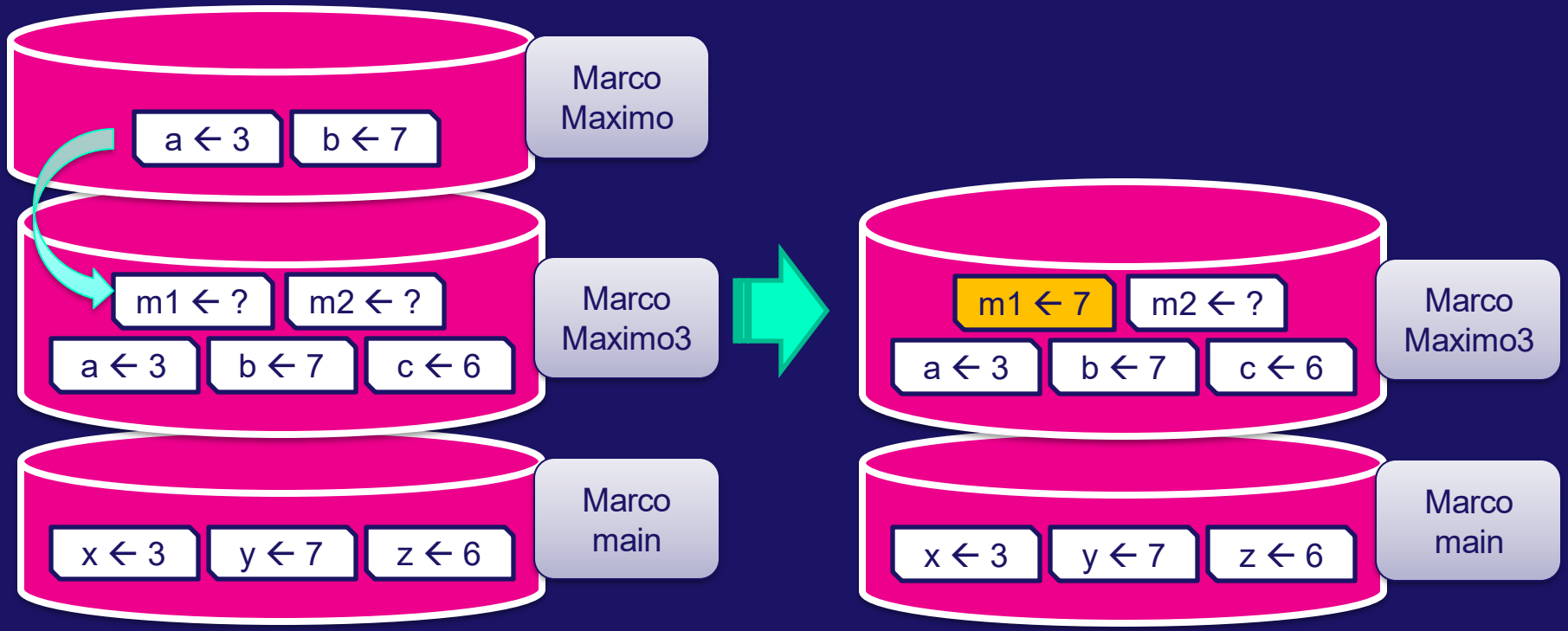


Primera llamada a Maximo

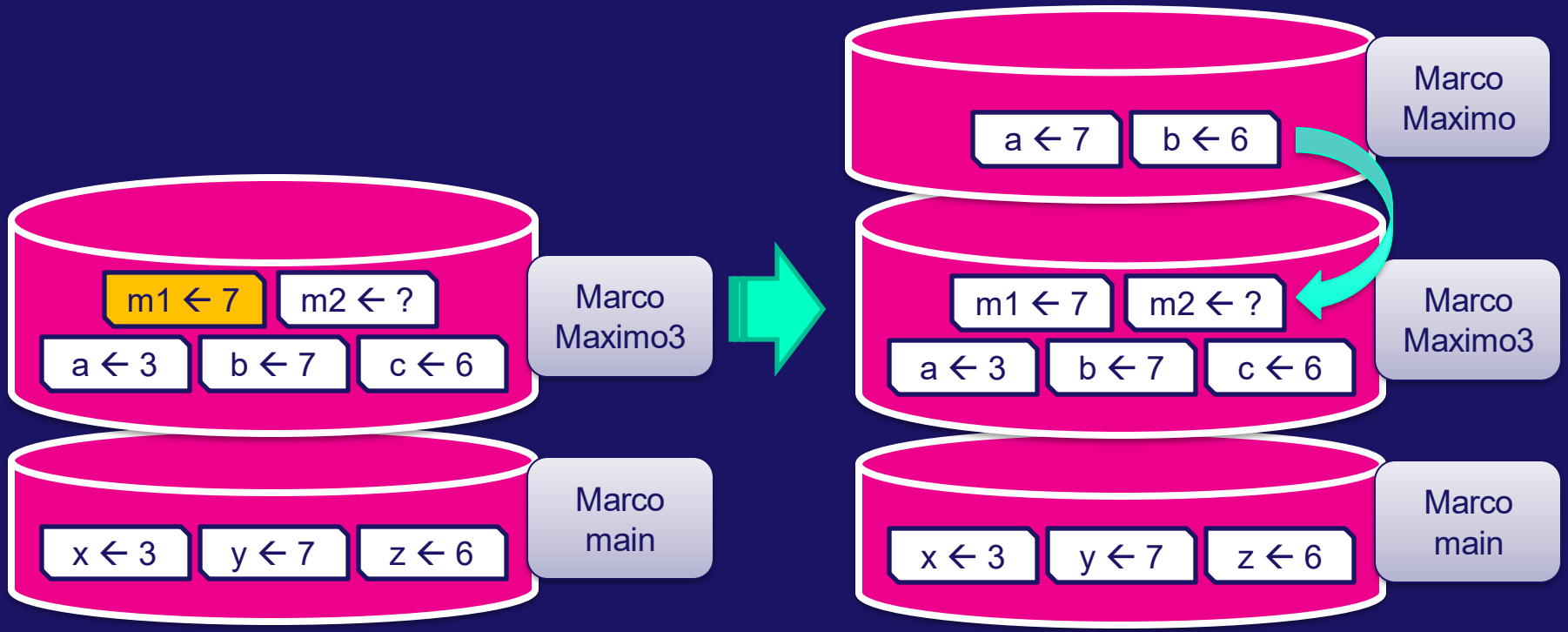
- Los contextos no solo almacenan las variables de la función (argumentos o locales)
- Almacenan la dirección (variable) en la que se asigna el valor del `return`
- El contexto de la función `Maximo` contiene la dirección de la variable m1 del contexto de `Maximo3`
- Cuando la función `Maximo` termina, se actualiza `m1` con el valor devuelto
- OJO: las variables a,b de `Maximo` y `Maximo3` tienen el mismo nombre, pero "viven" en contextos diferentes
  - No son las mismas variables



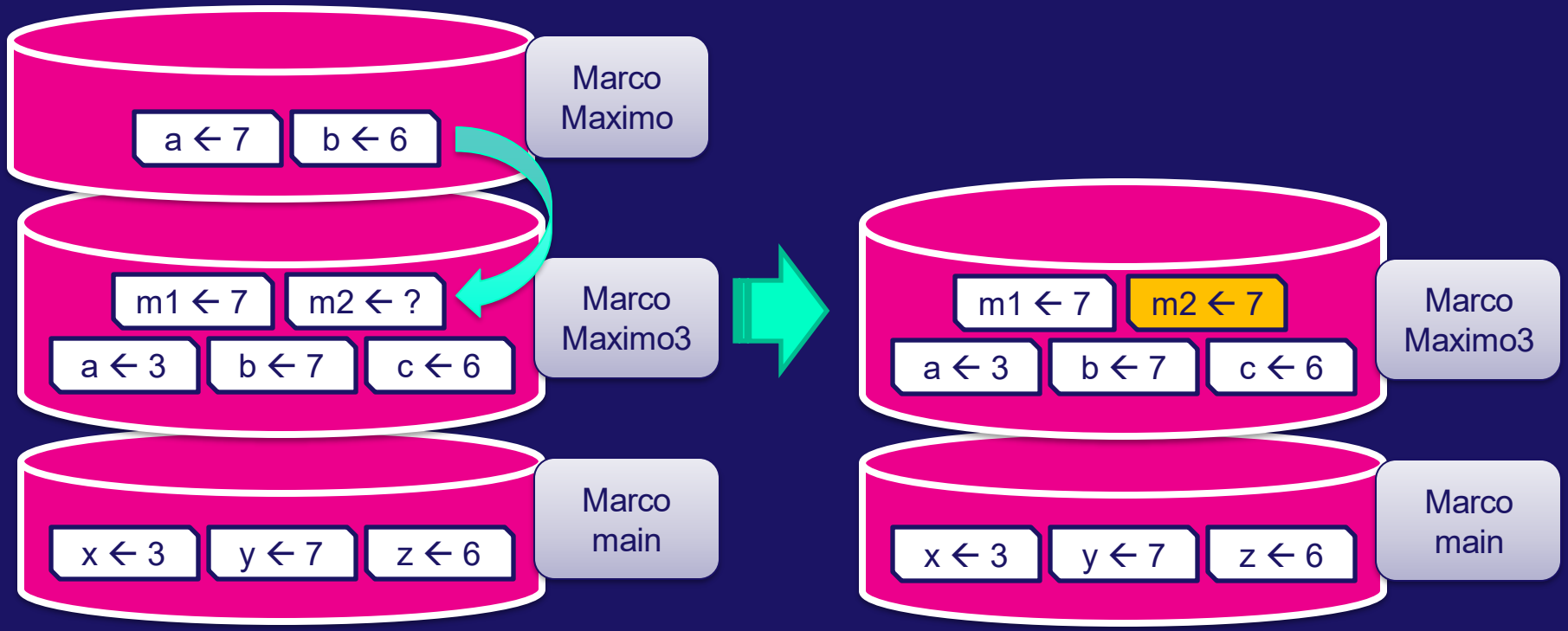
Primer return de Maximo



Segunda llamada a Maximo



Segundo return de Maximo

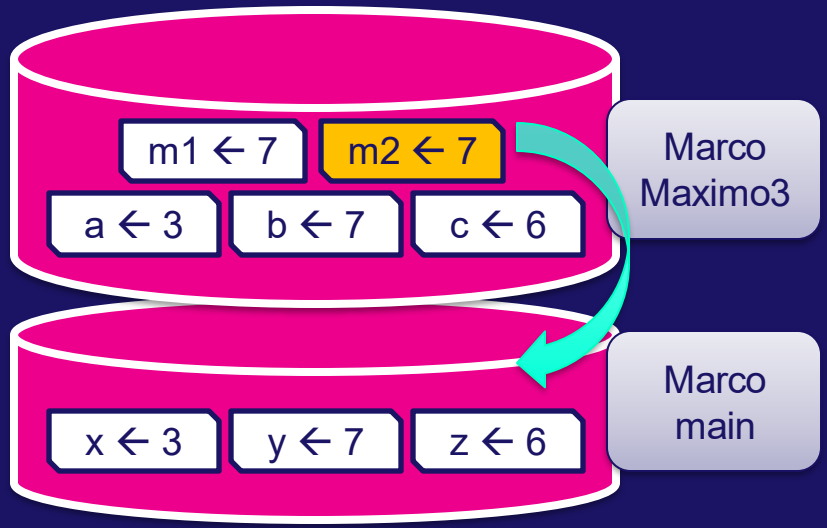


Desapilamiento de llamadas

- Una vez se ha terminado la segunda llamada a `Maximo`, solo queda desapilar el contexto de `Maximo3`

- El valor devuelto por Maximo3 se utiliza por el `printf` una vez el contexto de `Maximo3` se destruye
  - En ese momento, el contexto del `main` vuelve a estar activo

```
Salida de consola
7
```



Ejercicios

1. Defina una función que calcule el mínimo de dos valores.
2. Muestre como se puede utilizar la función anterior para calcular el mínimo de tres valores. ¿Y con cuatro?
3. Pedir número en un intervalo
4. Defina e implemente una función que informe si un número es feliz

Documentación de funciones

- Antes del prototipo se suele incluir un comentario con la descripción/documentación de la función
- Elementos de la documentación:
  - Cometido
  - Entradas
  - Salidas
  - Requisitos

Documentación de una función

```c
/**
 * Función: Max3
 * Cometido: Calcular el máximo de tres números enteros.
 * Entradas:
 *    - a: Primer número int a comparar.
 *    - b: Segundo número int a comparar.
 *    - c: Tercer número int a comparar.
 * Salida:  Devuelve el mayor de los tres números proporcionados como entrada (int).
 */
int Max3(int a, int b, int c) {
    int max;
    if (a >= b)
        max = a;
    else
        max = b;
    if (c > max)
        max = c;

    return max;
}
```

02

FUNCIONES VOID

Funciones void – Procedimientos

- Son módulos o “funciones” que no devuelven un valor
- Se pueden utilizar para:
  - Imprimir mensajes por printf solamente
  - Mostrar un menú de opciones
- La manera de indicar que no se devolverá nada es con el tipo “vacío” o *void*

```
void nombre_funcion(parametros formales){
    ...código de la funcion (cuerpo) ...
}
```

Funciones void – Procedimientos

```c
void MenuOpciones();
int main() {
    int opcion;
    MenuOpciones();
    scanf("%d", &opcion);
    /*Resto del programa*/
    return 0;
}

void MenuOpciones() {
    printf("1. Opcion 1 - sumar\n");
    printf("2. Opcion 2 - restar\n");
    printf("3. Opcion 3 - multiplicar\n");
    printf("4. Opcion 4 - dividir\n");
    printf("5. Opcion 5 - salir\n");
    printf("Seleccione una opcion: ");
}
```

Al llamar a un procedimiento, no podemos utilizar la asignación '=', ya que no se devuelve NADA

Ya que los procedimientos no devuelven nada, podemos obviar la sentencia *return*

Paso de parámetros por valor

- Es el mecanismo que hemos visto hasta ahora.
- El valor de los argumentos se copia en los parámetros formales.
- Los argumentos que se "pasan" por valor, también reciben el nombre de parámetros de entrada.
- Los parámetros formales funcionan como variables locales.
- Hemos visto que los argumentos pueden ser variables o valores literales: `Cuadrado(4), Cuadrado(n)`

Paso de parámetros por valor

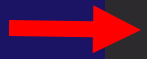
Mi Programa

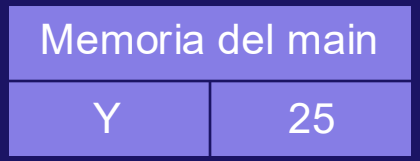
```
int main(){
 int Y, res;

 Y = 25;
 res = MiFuncion(Y);
}
```

MiFuncion

```
int MiFuncion(int x)
{
  instrucciones
}
```









- La función "MiFuncion" trabaja sobre una copia del valor del parámetro actual

Paso de parámetros por referencia

- Un parámetro por referencia sirve para que la función comunique valores calculados al programa que la invocó.
- Para usar parámetros por referencia el identificador del parámetro formal debe estar precedido por el símbolo *.
- **El parámetro real debe ser una variable, nunca una expresión ni un literal y debe ir precedida de &.**
- Los parámetros por referencia llevan un * delante cuando se usan en la función
- **Consejo → Si una función da como resultado más de un valor, entonces utilizaremos parámetros por referencia**
    - Caso típico → funciones que son casi idénticas en sus cálculos → fusionarlas

Paso de parámetros por referencia

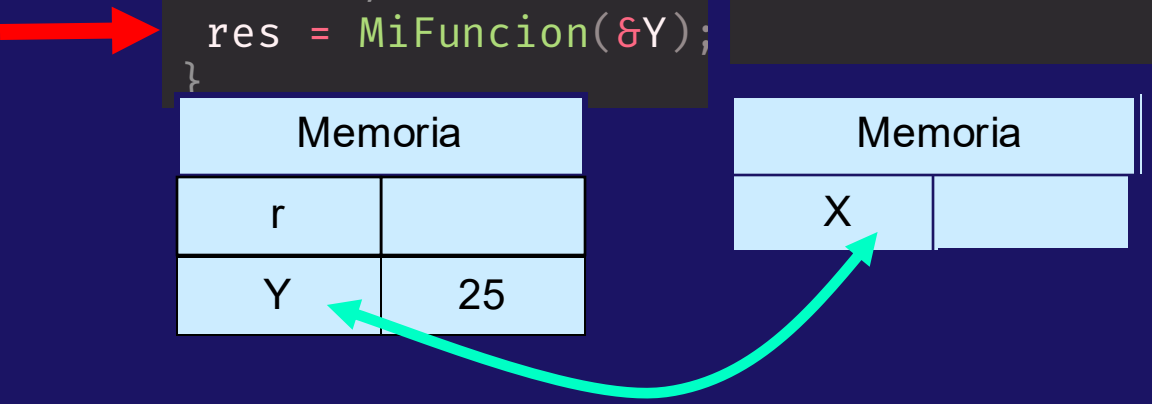
Mi Programa

```
int main(){
 int Y, res;

 Y = 25;
 res = MiFuncion(&Y);
}
```

miFuncion

```
int MiFuncion(int *x)
{
  instrucciones
}
```



- La variable x es un "alias" para la variable Y.
- Todos los cambios que hagamos dentro de la función "MiFuncion" sobre la variable x se reflejarán en el parámetro actual correspondiente (Y).

Ejemplo

```c
#include <stdio.h>
#include <math.h>

/* calcula area y perímetro de un circulo de radio "radio"  */
void AreaPeri(double radio, double *area, double *perimetro);

int main() {
    double r = 2.5;
    double a, p;

    AreaPeri(r, &a, &p);

    printf("Area: %.2lf, Perimetro: %.2lf\n", a, p);
    return 0;
}
```

```c
void AreaPeri(double radio, double *area, double *perimetro) {
    *area = M_PI * radio * radio;
    *perimetro = 2 * M_PI * radio;
}
```

Ejercicios

1. Implemente una función Ordena que reciba dos números y devuelva el menor en el primer argumento y el mayor en el segundo.
2. Usando la función del ejercicio anterior, implemente una función que reciba tres variables de tipo entero a, b y c, y los reasigne de forma tal que a <= b <= c
3. Implemente una función que modifique el valor de un ángulo en radianes a grados.

Ejercicios

4. Implemente una función que calcule el factorial de un número dado.
5. Implemente una función comb(n,m) que calcule el número combinatorio

$$\binom{m}{n} = \frac{m!}{n!(m-n)!}$$

6. Diseñe una función que invierta los dígitos de un número natural suministrado, así como el número de dígitos procesados.

03

DISEÑO DE FUNCIONES

Función main

- El módulo main es otra función
- Tiene un nombre especial, main, que lo identifica como la función principal:
  - La ejecución de cualquier programa consiste en ejecutar su contenido
  - Cualquier programa finaliza cuando termina la función main
- La función main es de tipo *int* → devuelve un entero con un código de error
  - El valor 0 indica que la función (el programa) ha terminado correctamente
  - Es la única función para la que se puede omitir el `return` → si no está, el compilador añade automáticamente `return(0);` justo antes de la llave }

Diseño modular

- Podemos hacer múltiples divisiones modulares para un mismo problema
- Tener más módulos NO significa tener una mejor división

- Lo ideal → dividir en subproblemas

- Al dividir en subproblemas, implementamos funciones como cajas negras

Diseño modular

- El usuario de la función debe saber QUÉ hace la función y no CÓMO lo hace.
- El hecho de que un módulo oculte sus detalles internos (el cómo) se denomina **ocultamiento de información**
- Los módulos deben diseñarse para hacer una única tarea concreta y no varias

Diseño modular

- Función que hace entrada de datos y los cálculos
- PROBLEMA: otra función que ya tenga un entero no puede usar nuestra función

```
int Factorial(){
    int n, resultado = 1, i;

    scanf("%d", &n);

    for(i=2; i<n; i++){
        resultado *= i;
    }
    return resultado;
}
```

Diseño modular

- Si la función solo hace el cálculo se puede reutilizar

- No mezclar E/S y cómputos en las funciones

```
int Factorial(int n){
    int i, resultado = 1;

    for(i=2; i<n; i++){
        resultado *= i;
    }

    return resultado;
}
```

Ámbito de las variables (2)

- ¡Podemos declarar variables fuera de las funciones!
- De esta forma podemos utilizar datos que sean “vistos” por todas nuestras funciones

```c
#include <stdio.h>

char letra; /* variable global */


void Aumenta(){
    letra++;
}
void Imprime(){
    printf("%c\n", letra);
}
int main(){
    letra = 'A';
    Imprime();
    Aumenta();
    Imprime();
}
```

Ámbito de las variables (2)

- El uso de las variables globales presenta varios problemas:
  1. Disminuye el ocultamiento de la información
  2. Dificulta MUCHO la depuración de los programas → todas las funciones que usen variables globales se influyen entre sí → aislar errores es difícil
  3. Una función que no opere correctamente puede modificar una variable global de forma incorrecta, generando errores en cascada en otras funciones que SÍ están bien

- Por ello debemos tener en cuenta lo siguiente

NUNCA USAREMOS NI SE PERMITIRÁ EL USO DE VARIABLES GLOBALES EN ESTA ASIGNATURA

Ámbito de las variables (2)

- Un uso que SÍ se permite del ámbito global es el uso de constantes globales
- Como no pueden cambiar su valor, no es posible provocar efectos colaterales

- No pueden ser modificadas por ningún módulo → no se pueden utilizar para enviar información entre módulos → no hay relaciones entre módulos usando constantes globales
- Ventaja: Con una modificación simple, cambio el valor de la constante en todos los módulos

Ámbito de las variables (2)

Definida fuera de la funcion “main”

```c
#include <stdio.h>
#include <math.h>

/* usar M_PI sería mejor… pero es un ejemplo*/
const double PI=3.14159;
/* calcula el area de un circulo de radio "radio“*/
double Area(double radio);
/* calcula el volumen de una esfera de radio "radio“*/
double Volumen(double radio);
int main (){
 double r = 2.5;
 printf("%f\n", Area(r));
 printf("%f\n", Volumen(r));
 return(0);
}
```

```c
double Area(double radio){
  return(PI*radio*radio);
}
double Volumen(double radio ){
  double tmp = pow(radio,3);
  return((4.0/3.0)*PI*tmp)
}
```

Ejercicios

Se dispone de un conjunto de medidas agrupadas por su clase, incluyendo las reglas de conversión.

Distancia	Peso	Temperatura	Volumen
1 metro	**1 kg.**	**1º Celsius**	**1 Litro**
3.28 pies	2.2 libras	274.15 Kelvin	0.22 galones
39.37 pulgadas	32.57 onzas	33.8 Farenheit	1.76 pintas

Diseñar un programa que permita al usuario elegir el tipo de medida, ingresar el valor de referencia y posteriormente la conversión que desea realizar.

El programa debe mostrar menús para las medidas y posibles conversiones. Debe considerar solamente entradas válidas y debe utilizar adecuadamente funciones y procedimientos

04

RECURSIVIDAD

Recursividad en matemáticas

- El concepto de recursividad aparece recurrentemente en matemáticas
- Muchas funciones matemáticas se definen en términos de sí mismas
- Ejemplo de función recursiva: el factorial

$$n! = \begin{cases} 1 & \text{si } n = 0 \\ n \cdot (n-1)! & \text{si } n > 0 \end{cases}$$

- También relacionado con la demostración por inducción

Recursión

- Podemos pensar de la misma forma en programación:
  - Si el problema se puede expresar en términos de sí mismo y
  - El problema está resuelto para un caso "pequeño" (base)
  - Entonces podemos expresar la solución de forma recursiva
- Para el factorial:
  - El caso base → n = 0 y factorial = 1
  - Si tenemos resuelto el factorial de (n-1), obtener el factorial de n se consigue multiplicando (n-1)! por n
- No siempre tendremos un algoritmo ya expresado de forma recursiva → tendremos que diseñarlo nosotros

Recursión directa e indirecta

- Cuando una función recursiva se utiliza "a sí misma" directamente nos encontramos con recursión directa
- Hay ocasiones en las que una función A utiliza otra función B, y B utiliza la función A original.
  - Este tipo de recursión se llama indirecta
  - Puede generar esquemas de llamadas y utilizaciones muy complejos
- En esta asignatura nos centraremos en recursión directa

Factorial de un número

- Recordemos el caso base y la definición recursiva del factorial

$$n! = \begin{cases} 1 & \text{si } n = 0 \\ n \cdot (n-1)! & \text{si } n > 0 \end{cases}$$

```
int Factorial(int n){
    if(n==0)
        return 1;
    else
        return n * Factorial(n-1);
}
```

Caso base

Recursión

Funcionamiento de las llamadas

- El funcionamiento de las llamadas a las funciones no difiere de lo visto en el Tema 3

- Cada vez que se hace una llamada recursiva, se copia el valor de n-1 en la nueva función (en la pila)
  - Si vale 0, se devuelve 1 y se "deshacen" las llamadas
  - Si vale > 0, se repite la copia de n - 1 y se "**apila" una nueva llamada**

Funcionamiento de las llamadas

```
int Factorial(int n){
    if(n==0)
        return 1;
    else
        return n * Factorial(n-1);
}
```

"Aparcado"

n ← 2



```
printf("%d\n", Factorial(2));
```

```
int Factorial(int n){
    if(n==0)
        return 1;
    else
        return n * Factorial(n-1);
}
```

"Aparcado"

n ← 1

```
int Factorial(int n){
    if(n==0)
        return 1;
    else
        return n * Factorial(n-1);
}
```

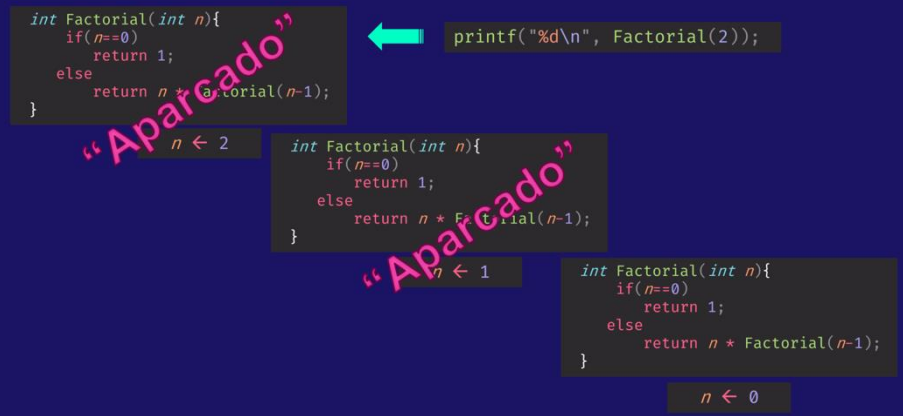
n ← 0

Funcionamiento de los *return*



Pila y recursividad

- La pila actúa de la misma forma en llamadas recursivas
- En el ejemplo del Factorial:
  - El texto Aparcado indica que hay un contexto por encima en la pila activo
  - La desaparición de elementos indica que se extraen de la pila



Pila y recursividad

- Importante: antes de llegar al caso base, tenemos varios contextos apilados
  - Cuando se hace el return del caso base es cuando se concretan los cálculos y se desapilan
- Profundidad de recursión = cuántos contextos se apilan para resolver un caso específico
- Por ejemplo, `Factorial(5)` → 6 contextos apilados cuando llegamos al caso base
- Nota: los contextos NO contienen el código de la función (sólo las variables)

Recursos

- Normalmente, la recursividad requiere más tiempo y memoria que una solución iterativa
- El uso de la recursividad tiene sentido en problemas complejos donde:
  - Expresar la solución de forma iterativa NO es sencillo
  - El número de llamadas (y por tanto, contextos) no es excesivo
- ¡Es importante no repetir llamadas para resolver el mismo subproblema!
  - Piense en el caso de la resolución recursiva de la función de Fibonacci